
Project Charter

for

PhytoPi-ESP32

Version 1.0 approved

Prepared by Nolan Tuttle, Daniel Grijalva

Software Engineering II

1/25/26

Table of Contents

1. Version Control	2
2. Introduction	3
2.1 Project Background and Rationale	3
2.2 Scope	3
2.3 Project Objectives	4
2.4 Release Versioning Management Process	4
2.5 Project Organization Chart	4
2.6 Project Governance	5
2.7 System Sensors and Data Sources	5

Github Repository & Project

Repository: <https://github.com/nolantuttle/PhytoPi-ESP32>

Project: <https://github.com/users/nolantuttle/projects/4>

Revision History

Name	Date	Reason For Changes	Version
Nolan Tuttle	1/25/26	Initial Document Creation	1.0

Member Name	Email	Team Role	Signatures
Daniel Grijalva	Dgrijalva3@my.gcu.edu	Software + Cloud Lead	
Nolan Tuttle	NTuttle1@my.gcu.edu	Hardware + Integration Lead	

1. Version Control

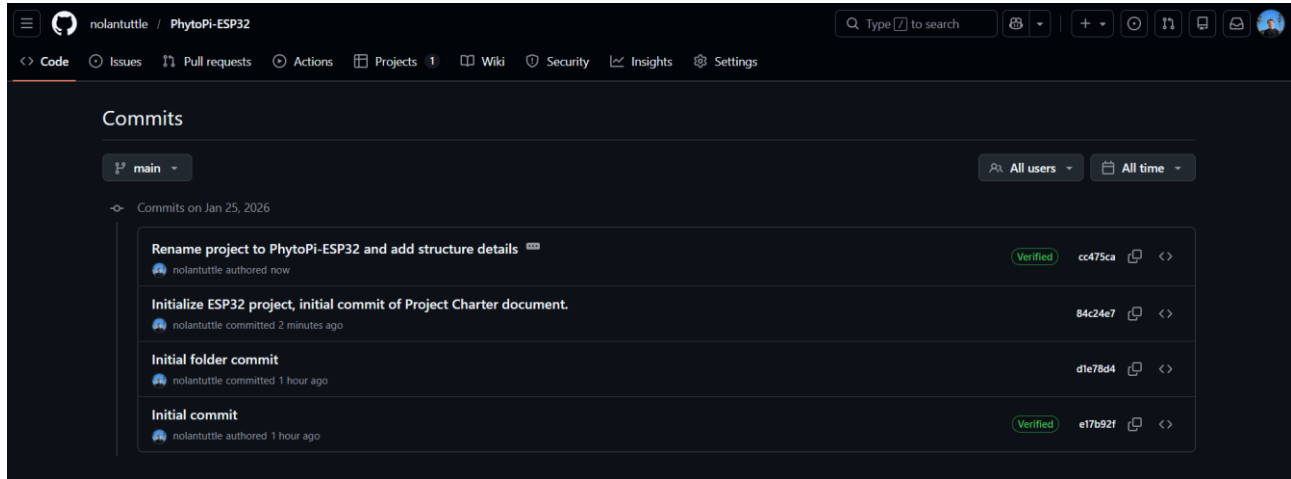


Figure 1: Screenshot of a Pending Commit to Github using the Visual Studio Code IDE

Figure 2: Screenshot Showing the Commit History on the 'main' branch for the Github Repository



Figure 3: Screenshot of the Project Branches, a 'main' branch for Production-ready Features and a 'feature' branch for Feature Testing Before Pushing to 'main'

2. Introduction

This project proposed is the PhytoPi-ESP32 plant monitoring system. The purpose of this project is to achieve automation of repetitive plant care tasks and sustain most needs of plant growth throughout a plant's lifespan. The project is designed in such a way that it caters specifically to the needs of basil plants to limit the scope of the project and to present a proof-of-concept for further improvements.

2.1 Project Background and Rationale

PhytoPi is an IoT-based controlled environment system for supporting a plant throughout its entire growth cycle with minimal human intervention. Utilizing embedded hardware and a machine learning system monitoring the plant using a camera, PhytoPi is designed to automatically care for more than 80% of a plant's required maintenance over its lifespan. The goal upon completion of a minimum viable product is to support at least one species of plant throughout its growth cycle from germination to harvest/maturity, or for at least for 70 days.

2.2 Scope

The scope of this project is limited by many variables, most importantly of which is that the objective of the project is not to achieve complete automation of plant care. Due to the nature of conventional soil-based plant growth, acidity balance, disease, and other factors impacting plant growth cannot be fully accounted for with an affordable IoT solution, thus it is out of the scope of the project. This project seeks to automate over 80% of a typical basil plant's life cycle, with little human intervention required for plant maintenance. The scope of the project also does not include plants besides basil because of the introduction of changing plant growth styles, it creates too many variables to be controlled and thus, weakens the intended proof-of-concept being achieved.

2.3 Project Objectives

The PhytoPi-ESP32 project seeks to solve two key issues; the consistent maintenance requirements of plant care and the difficulties in data collection faced when one main controller must handle all database actions, UI display, and data collection utilizing its single processor. To address these issues, PhytoPi-ESP32 seeks to implement various IoT data collection devices and an ESP32 microcontroller to streamline data collection of plant health data, allowing decisions about watering to be made in order to automate plant care. The ESP32 will transmit collected data over Bluetooth Low Energy (BLE) protocols to eliminate required wiring and simplify data handling for the main Raspberry Pi 5 microprocessor.

2.4 Release Versioning Management Process

- Version Scheme
 - Semantic Versioning is used for this project (MAJOR.MINOR.PATCH)
 - Example: v.1.0.0 would be the first major release without any minor or patch releases.
- Version Incrementation
 - MAJOR version incremented for significant architectural changes or breaking functionality
 - MINOR version incremented when new features are added without breaking existing functionality
 - PATCH version incremented for bug fixes or minor improvements
- Releases
 - Releases are created from the main branch only in a deployable state

2.5 Project Organization Chart



2.6 Project Governance

1. Decision-Making

- a. Project Team: Responsible for day-to-day decisions on project design and development.
 - b. Professor/Mentor: Consultant for technical decisions, help to make project compliant with university standards
2. Communication and Check-ins
 - a. Weekly Class Check-ins
 - i. Stand-up style status updates
 - ii. Report will include progress, current tasks/delays
 - b. Weekly Team Meetings
 - i. In-person or video meetings
 - ii. Scheduled for every Wednesday at 1:00PM MST
 - iii. Review Github project board and weekly tasks/issues
3. Change Management and QA
 - a. Proposed Changes
 - i. Created as Github Issues in backlog
 - ii. Development work is performed in feature branch and tested before any changes to main branch
 - b. Quality Assurance
 - i. Pull request review before merging with main branch
 - ii. Main branch MUST remain in a deployable state

2.7 System Sensors and Data Sources

To support the defined project scope and objectives, the system utilizes environmental sensors connected to an ESP32 to collect data required for monitoring plant health and automating plant care decisions.

- Capacitive Soil Moisture Sensor
 - Measures soil moisture levels without direct electrical contact with the soil
 - Used to determine when automated watering is required
 - Supports the objective of reducing manual plant maintenance
- Temperature and Humidity Sensor
 - Measures ambient air temperature and relative humidity
 - Used to monitor environmental conditions that affect plant growth
 - Data supports environmental trend analysis and system monitoring
- Data Collection and Transmission
 - Sensor data is collected by the ESP32
 - Data is transmitted to the Raspberry Pi using Bluetooth Low Energy (BLE)
 - The Raspberry Pi processes, stores, and presents data through a web-based dashboard

Reference

N/A